Taller 6: Capítulo 20

Cubos Multidimensionales

Derly Mahecha y Kevin Reyes

Tabla de contenidos

1 Introducción 2

2 Lectura del archivo NetCDF con xarray y dask 2

3 Verificación de la proyección espacial con rioxarray 3

4 Conversión de kelvin a Celsius y agregación temporal 3

5 Graficar media como mapa de pixeles 4

6 Verificación de unidades actuales en Celsius 5

7 Graficar variable convertida como mapa de pixeles 6

8 Crear un índice de tiempo con pandas para verificar las fechas 7

9 Calculamos la diferencia entre los dos primeros registros de tiempo 8

10 Remuestreo temporal con xarray (Resampling) 8

11 Calculamos el valor máximo mensual para cada píxel 9

12 Calculamos la fecha del valor máximo mensual para cada píxel 10

13 Visualizamos la fecha del valor máximo mensual para cada píxel 11

14 Exportamos el resultado a un nuevo archivo NetCDF 13

15 Respuestas del Capítulo 20: Cubos multidimensionales 13

1. Introducción

Los "cubos multidimensionales" son estructuras de datos que organizan información en múltiples dimensiones, como tiempo, espacio y variables. En el contexto de datos geoespaciales, un cubo multidimensional puede contener capas de información para diferentes momentos en el tiempo, ubicaciones geográficas y variables climáticas o ambientales. Estos cubos permiten realizar análisis complejos y visualizaciones avanzadas, facilitando la comprensión de patrones y tendencias en los datos.

2. Lectura del archivo NetCDF con xarray y dask

```python
import os
import xarray as xr

nc_file = r"E:\ProgramacionSIG\data_heavy\era5_2m_temperature_2025_04_colombia.nc"
if os.path.exists(nc_file):
    ds = xr.open_dataset(nc_file, chunks="auto")  # lectura lazy con dask
    print("Estructura del dataset:")
    print(ds)
    print(f"\nDimensiones: {dict(ds.sizes)}")
    print(f"Variables: {list(ds.data_vars)}")
else:
    print(f"Error: no existe {nc_file}")
```

```
Estructura del dataset:
<xarray.Dataset> Size: 89kB
Dimensions:     (valid_time: 720, latitude: 5, longitude: 5)
Coordinates:
  * valid_time  (valid_time) datetime64[ns] 6kB 2025-04-01 ... 2025-04-
30T23:...
    expver      (valid_time) <U4 12kB dask.array<chunksize=(720,), meta=np.ndarray>
  * latitude    (latitude) float64 40B 4.0 3.75 3.5 3.25 3.0
  * longitude   (longitude) float64 40B -75.0 -74.75 -74.5 -74.25 -74.0
    number      int64 8B ...
Data variables:
    t2m         (valid_time, latitude, longitude) float32 72kB dask.array<chunksize=(720, 5,
Attributes:
    GRIB_centre:             ecmf
    GRIB_centreDescription:  European Centre for Medium-Range Weather Forecasts
```

```
    GRIB_subCentre:          0
    Conventions:             CF-1.7
    institution:             European Centre for Medium-Range Weather Forecasts
    history:                 2026-04-25T20:32 GRIB to CDM+CF via cfgrib-
0.9.1...

Dimensiones: {'valid_time': 720, 'latitude': 5, 'longitude': 5}
Variables: ['t2m']
```

3. Verificación de la proyección espacial con rioxarray

```python
import rioxarray

ds = ds.rio.set_spatial_dims(x_dim="longitude", y_dim="latitude", inplace=False)
if ds.rio.crs is None:
    ds = ds.rio.write_crs("EPSG:4326", inplace=False)

print(f"CRS actual: {ds.rio.crs}")
```

```
CRS actual: GEOGCS["WGS 84",DATUM["World Geodetic System 1984",SPHEROID["WGS 84",6378137,298
```

4. Conversión de kelvin a Celsius y agregación temporal

```python
temp_var = "t2m" if "t2m" in ds.data_vars else ("t" if "t" in ds.data_vars else "tp")
time_dim = "valid_time" if "valid_time" in ds.dims else "time"
is_temp = temp_var in ["t2m", "t"]

da_conv = ds[temp_var] - 273.15 if is_temp else ds[temp_var] * 1000
da_conv.attrs["units"] = "°C" if is_temp else "mm"
agg = "mean" if is_temp else "sum"

t_mean = da_conv.mean(dim=time_dim)
print(f"Variable: {temp_var} | Unidad: {da_conv.attrs['units']} | Agregacion diaria: {agg}")
print(t_mean)
```

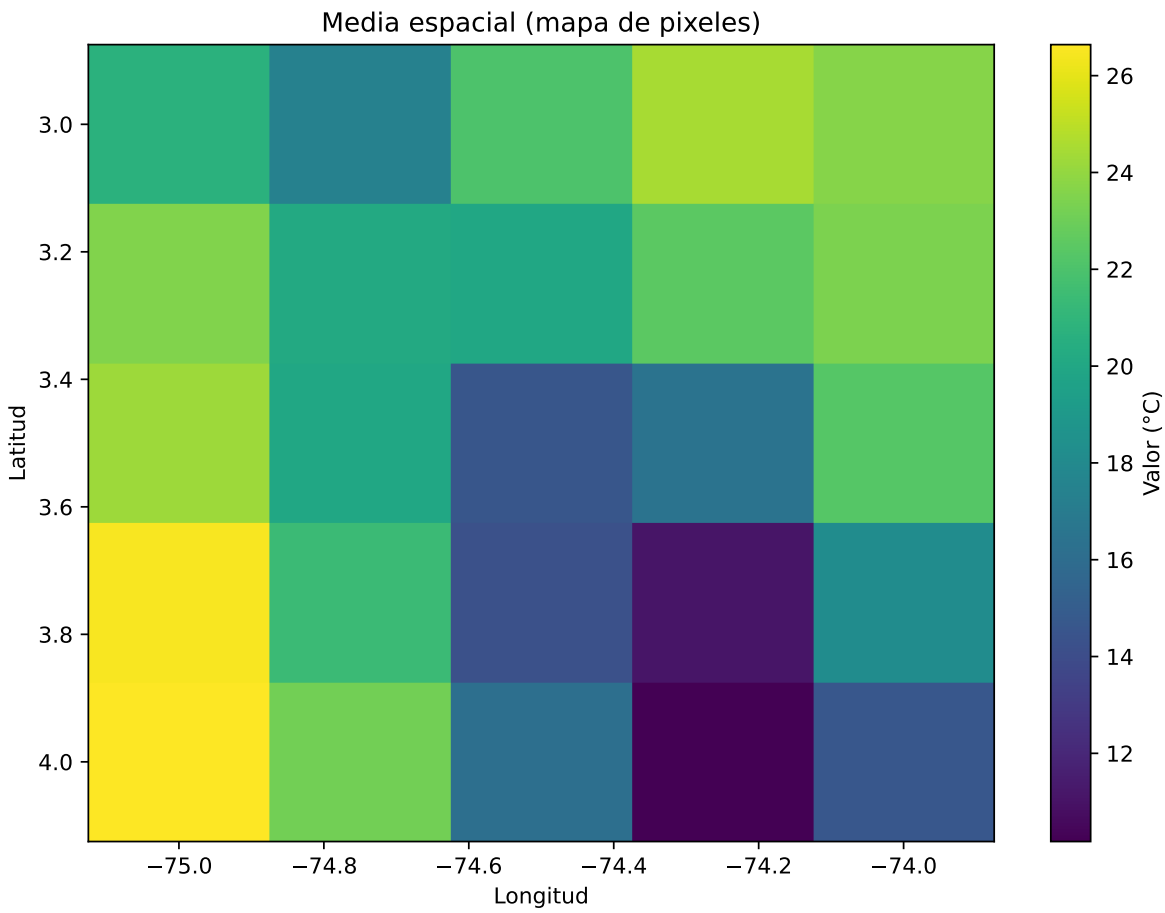
```
Variable: t2m | Unidad: °C | Agregacion diaria: mean
<xarray.DataArray 't2m' (latitude: 5, longitude: 5)> Size: 100B
```

```
dask.array<mean_agg-aggregate, shape=(5, 5), dtype=float32, chunksize=(5, 5), chunktype=numpy
Coordinates:
  * latitude     (latitude) float64 40B 4.0 3.75 3.5 3.25 3.0
  * longitude    (longitude) float64 40B -75.0 -74.75 -74.5 -74.25 -74.0
    number       int64 8B ...
    spatial_ref  int64 8B 0
Attributes: (12/32)
    GRIB_paramId:                           167
    GRIB_dataType:                          an
    GRIB_numberOfPoints:                    25
    GRIB_typeOfLevel:                       surface
    GRIB_stepUnits:                         1
    GRIB_stepType:                          instant
    ...                                     ...
    GRIB_totalNumber:                       0
    GRIB_units:                             K
    long_name:                              2 metre temperature
    units:                                  °C
    standard_name:                          unknown
    GRIB_surface:                           0.0
```

5. Graficar media como mapa de pixeles

```python
import matplotlib.pyplot as plt

da_plot = t_mean.squeeze()
dims_to_reduce = [d for d in da_plot.dims if d not in ("latitude", "longitude")]
if dims_to_reduce:
    da_plot = da_plot.mean(dim=dims_to_reduce)

if da_plot.ndim == 2:
    plt.figure(figsize=(8, 6))
    da_plot.plot.imshow(
        cmap="viridis",
        interpolation="nearest",
        yincrease=False,
        cbar_kwargs={"label": f"Valor ({da_conv.attrs.get('units', '')})"}
    )
    plt.title("Media espacial (mapa de pixeles)")
    plt.xlabel("Longitud")
```

```
    plt.ylabel("Latitud")
    plt.tight_layout()
    plt.show()
```



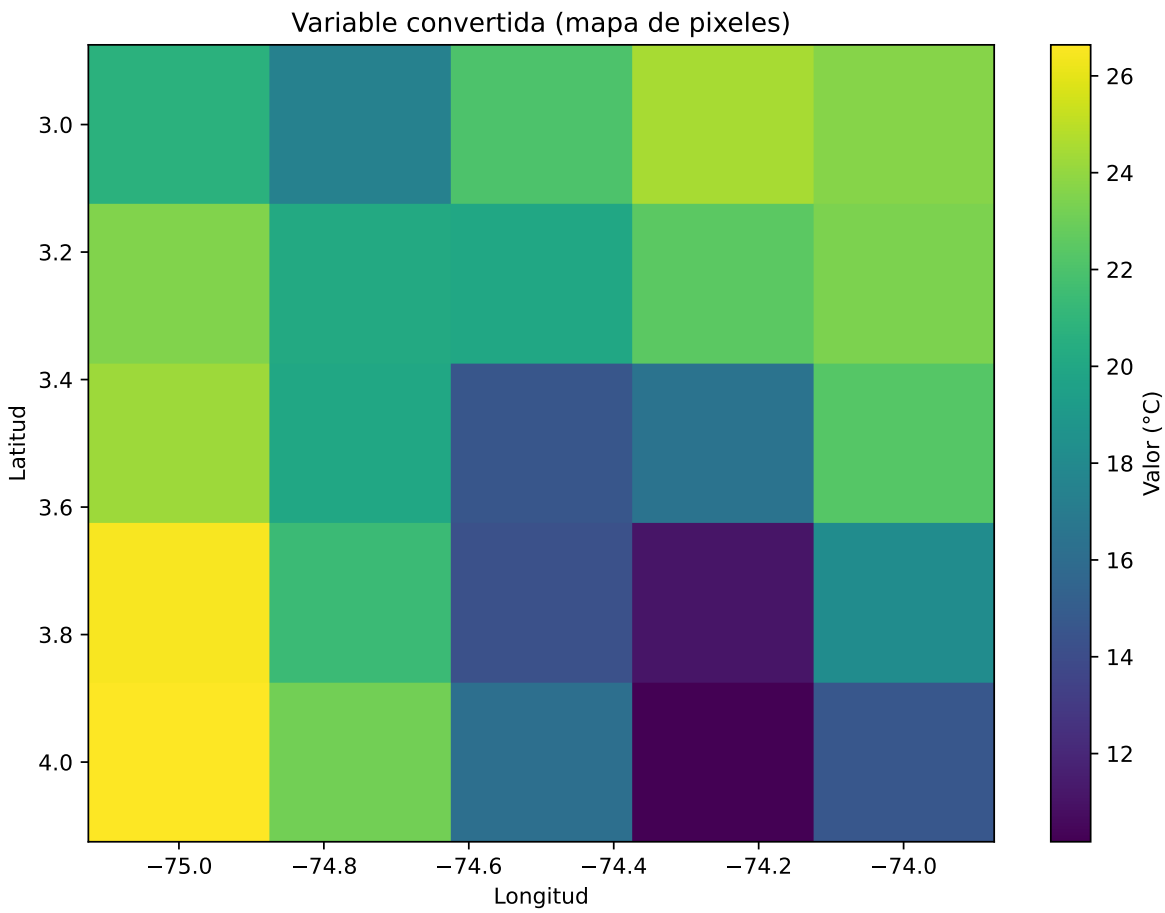
6. Verificación de unidades actuales en Celsius

```
t_celsius = da_conv
print(f"Unidad actual: {t_celsius.attrs.get('units')}")
```

```
Unidad actual: °C
```

7. Graficar variable convertida como mapa de pixeles

```
import matplotlib.pyplot as plt

da_plot = t_celsius.squeeze()
dims_to_reduce = [d for d in da_plot.dims if d not in ("latitude", "longitude")]
if dims_to_reduce:
    da_plot = da_plot.mean(dim=dims_to_reduce)

if da_plot.ndim == 2:
    plt.figure(figsize=(8, 6))
    da_plot.plot.imshow(
        cmap="viridis",
        interpolation="nearest",
        yincrease=False,
        cbar_kwargs={"label": f"Valor ({t_celsius.attrs.get('units', '')})"}
    )
    plt.title("Variable convertida (mapa de pixeles)")
    plt.xlabel("Longitud")
    plt.ylabel("Latitud")
    plt.tight_layout()
    plt.show()
```



8. Crear un índice de tiempo con pandas para verificar las fechas

```
import pandas as pd
time_index = pd.to_datetime(ds[time_dim].values)
print(time_index[:5], "...", time_index[-1])
```

```
DatetimeIndex(['2025-04-01 00:00:00', '2025-04-01 01:00:00',
               '2025-04-01 02:00:00', '2025-04-01 03:00:00',
               '2025-04-01 04:00:00'],
              dtype='datetime64[ns]', freq=None) ... 2025-04-30 23:00:00
```

9. Calculamos la diferencia entre los dos primeros registros de tiempo

```
delta_time = time_index[1] - time_index[0]
print(f"Resolución temporal: {delta_time}")
```

```
Resolución temporal: 0 days 01:00:00
```

10. Remuestreo temporal con xarray (Resampling)

```
t_diaria = t_celsius.resample({time_dim: "1D"}).mean()
print("\n--- Temperatura diaria promedio ---")
print(t_diaria)
print("\n--- Tiempo original ---")
print(t_celsius[time_dim])
```

```
--- Temperatura diaria promedio ---
<xarray.DataArray 't2m' (valid_time: 30, latitude: 5, longitude: 5)> Size: 3kB
dask.array<stack, shape=(30, 5, 5), dtype=float32, chunksize=(1, 5, 5), chunktype=numpy.ndar
Coordinates:
  * valid_time   (valid_time) datetime64[ns] 240B 2025-04-01 ... 2025-04-30
  * latitude     (latitude) float64 40B 4.0 3.75 3.5 3.25 3.0
  * longitude    (longitude) float64 40B -75.0 -74.75 -74.5 -74.25 -74.0
    number       int64 8B ...
    spatial_ref  int64 8B 0
Attributes: (12/32)
    GRIB_paramId:                            167
    GRIB_dataType:                           an
    GRIB_numberOfPoints:                     25
    GRIB_typeOfLevel:                        surface
    GRIB_stepUnits:                          1
    GRIB_stepType:                           instant
    ...                                      ...
    GRIB_totalNumber:                        0
    GRIB_units:                              K
    long_name:                               2 metre temperature
```

```
    units:                                      °C
    standard_name:                              unknown
    GRIB_surface:                               0.0

--- Tiempo original ---
<xarray.DataArray 'valid_time' (valid_time: 720)> Size: 6kB
array(['2025-04-01T00:00:00.000000000', '2025-04-01T01:00:00.000000000',
       '2025-04-01T02:00:00.000000000', ..., '2025-04-30T21:00:00.000000000',
       '2025-04-30T22:00:00.000000000', '2025-04-30T23:00:00.000000000'],
      shape=(720,), dtype='datetime64[ns]')
Coordinates:
  * valid_time   (valid_time) datetime64[ns] 6kB 2025-04-01 ... 2025-04-
30T23...
    expver       (valid_time) <U4 12kB dask.array<chunksize=(720,), meta=np.ndarray>
    number       int64 8B ...
    spatial_ref  int64 8B 0
Attributes:
    long_name:      time
    standard_name:  time
```

11. Calculamos el valor máximo mensual para cada píxel

```
t_mensual_max = t_diaria.resample({time_dim: "1M"}).max()
print("\n--- Temperatura máxima mensual ---")
print(t_mensual_max)
```

```
--- Temperatura máxima mensual ---
<xarray.DataArray 't2m' (valid_time: 1, latitude: 5, longitude: 5)> Size: 100B
dask.array<stack, shape=(1, 5, 5), dtype=float32, chunksize=(1, 5, 5), chunktype=numpy.ndarra
Coordinates:
  * valid_time   (valid_time) datetime64[ns] 8B 2025-04-30
  * latitude     (latitude) float64 40B 4.0 3.75 3.5 3.25 3.0
  * longitude    (longitude) float64 40B -75.0 -74.75 -74.5 -74.25 -74.0
    number       int64 8B ...
    spatial_ref  int64 8B 0
Attributes: (12/32)
    GRIB_paramId:                               167
    GRIB_dataType:                              an
    GRIB_numberOfPoints:                        25
```

```
    GRIB_typeOfLevel:                          surface
    GRIB_stepUnits:                            1
    GRIB_stepType:                             instant
    ...                                        ...
    GRIB_totalNumber:                          0
    GRIB_units:                                K
    long_name:                                 2 metre temperature
    units:                                     °C
    standard_name:                             unknown
    GRIB_surface:                              0.0
```

12. Calculamos la fecha del valor máximo mensual para cada píxel

```python
def fecha_max_mensual(x):
    x = x.compute()
    return x.idxmax(dim=time_dim)

t_mensual_fecha_max = t_diaria.resample({time_dim: "1MS"}).map(fecha_max_mensual)
print("\n--- Fecha del maximo mensual ---")
print(t_mensual_fecha_max)
```

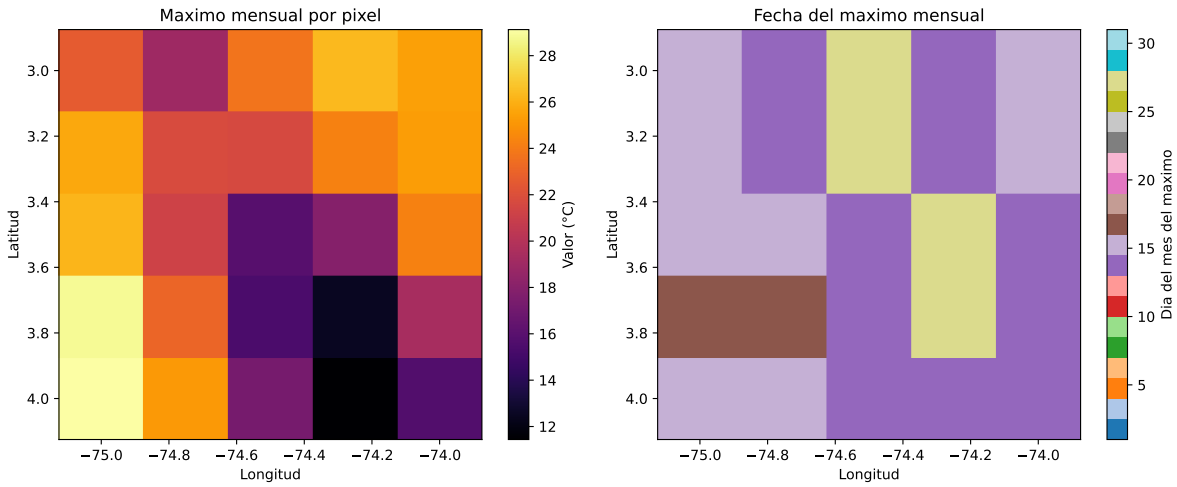
```
--- Fecha del maximo mensual ---
<xarray.DataArray 'valid_time' (valid_time: 1, latitude: 5, longitude: 5)> Size: 200B
array([[['2025-04-15T00:00:00.000000000',
         '2025-04-15T00:00:00.000000000',
         '2025-04-13T00:00:00.000000000',
         '2025-04-13T00:00:00.000000000',
         '2025-04-13T00:00:00.000000000'],
        ['2025-04-16T00:00:00.000000000',
         '2025-04-16T00:00:00.000000000',
         '2025-04-13T00:00:00.000000000',
         '2025-04-27T00:00:00.000000000',
         '2025-04-13T00:00:00.000000000'],
        ['2025-04-15T00:00:00.000000000',
         '2025-04-15T00:00:00.000000000',
         '2025-04-13T00:00:00.000000000',
         '2025-04-27T00:00:00.000000000',
         '2025-04-14T00:00:00.000000000'],
        ['2025-04-15T00:00:00.000000000',
```

```
         '2025-04-13T00:00:00.000000000',
         '2025-04-27T00:00:00.000000000',
         '2025-04-13T00:00:00.000000000',
         '2025-04-15T00:00:00.000000000'],
        ['2025-04-15T00:00:00.000000000',
         '2025-04-13T00:00:00.000000000',
         '2025-04-27T00:00:00.000000000',
         '2025-04-13T00:00:00.000000000',
         '2025-04-15T00:00:00.000000000']]], dtype='datetime64[ns]')
Coordinates:
  * valid_time   (valid_time) datetime64[ns] 8B 2025-04-01
  * latitude     (latitude) float64 40B 4.0 3.75 3.5 3.25 3.0
  * longitude    (longitude) float64 40B -75.0 -74.75 -74.5 -74.25 -74.0
    number       int64 8B 0
    spatial_ref  int64 8B 0
Attributes: (12/32)
    GRIB_paramId:                            167
    GRIB_dataType:                           an
    GRIB_numberOfPoints:                     25
    GRIB_typeOfLevel:                        surface
    GRIB_stepUnits:                          1
    GRIB_stepType:                           instant
    ...                                      ...
    GRIB_totalNumber:                        0
    GRIB_units:                              K
    long_name:                               2 metre temperature
    units:                                   °C
    standard_name:                           unknown
    GRIB_surface:                            0.0
```

13. Visualizamos la fecha del valor máximo mensual para cada píxel

```python
import matplotlib.pyplot as plt

max_plot = t_mensual_max.squeeze()
fecha_plot = t_mensual_fecha_max.squeeze()
dia_max = fecha_plot.dt.day

fig, ax = plt.subplots(1, 2, figsize=(12, 5))
max_plot.plot.imshow(
```

```
    ax=ax[0],
    cmap="inferno",
    yincrease=False,
    cbar_kwargs={"label": f"Valor ({t_celsius.attrs.get('units', '')})"}
 )
ax[0].set_title("Maximo mensual por pixel")
ax[0].set_xlabel("Longitud")
ax[0].set_ylabel("Latitud")

dia_max.plot.imshow(
    ax=ax[1],
    cmap="tab20",
    vmin=1, vmax=31,
    yincrease=False,
    cbar_kwargs={"label": "Dia del mes del maximo"}
 )
ax[1].set_title("Fecha del maximo mensual")
ax[1].set_xlabel("Longitud")
ax[1].set_ylabel("Latitud")

plt.tight_layout()
plt.show()
```



14. Exportamos el resultado a un nuevo archivo NetCDF

```
t_mensual_max.to_netcdf("mi_zona_procesada.nc")
print("Archivo 'mi_zona_procesada.nc' exportado exitosamente.")
```

```
Archivo 'mi_zona_procesada.nc' exportado exitosamente.
```

15. Respuestas del Capítulo 20: Cubos multidimensionales

1. Sobre Evaluación Perezosa (Lazy Evaluation): Imaginen que olvidaron hacer el remuestreo temporal a medias diarias y deciden reproyectar (hacer warping) el cubo horario completo de Copernicus a un sistema local (ej. EPSG:9377) para luego guardarlo. ¿Qué implicaciones tendría esta decisión en la memoria RAM de su computadora y en los tiempos de procesamiento?

Un cubo horario de Copernicus (por ejemplo ERA5 o S2) con resolución nativa de ~10–30 km tiene ~8.760 bandas temporales. En su proyección original (WGS84 o similar), los píxeles son grandes y la grilla es compacta: típicamente el archivo ocupa entre 20 y 80 GB en disco. Cuando se reproyecta directamente a EPSG:9377 (el sistema oficial de Colombia), ocurren tres cosas simultáneamente:

1. Aumento de resolución efectiva: El sistema proyectado local exige píxeles métricos más finos para preservar exactitud posicional, lo que multiplica el número de celdas por varios órdenes de magnitud.

2. Vecindad de interpolación en RAM: Cualquier método distinto de nearest neighbor (bilineal, bicúbico) necesita cargar bloques vecinos completos para calcular cada píxel de salida, lo que impide el procesamiento puramente secuencial.

3. Multiplicación por el eje temporal: La reproyección se aplica independientemente a cada una de las 8.760 bandas, de modo que el volumen resultante puede alcanzar fácilmente 300–500 GB que deben residir en RAM (o en swap) durante el proceso.

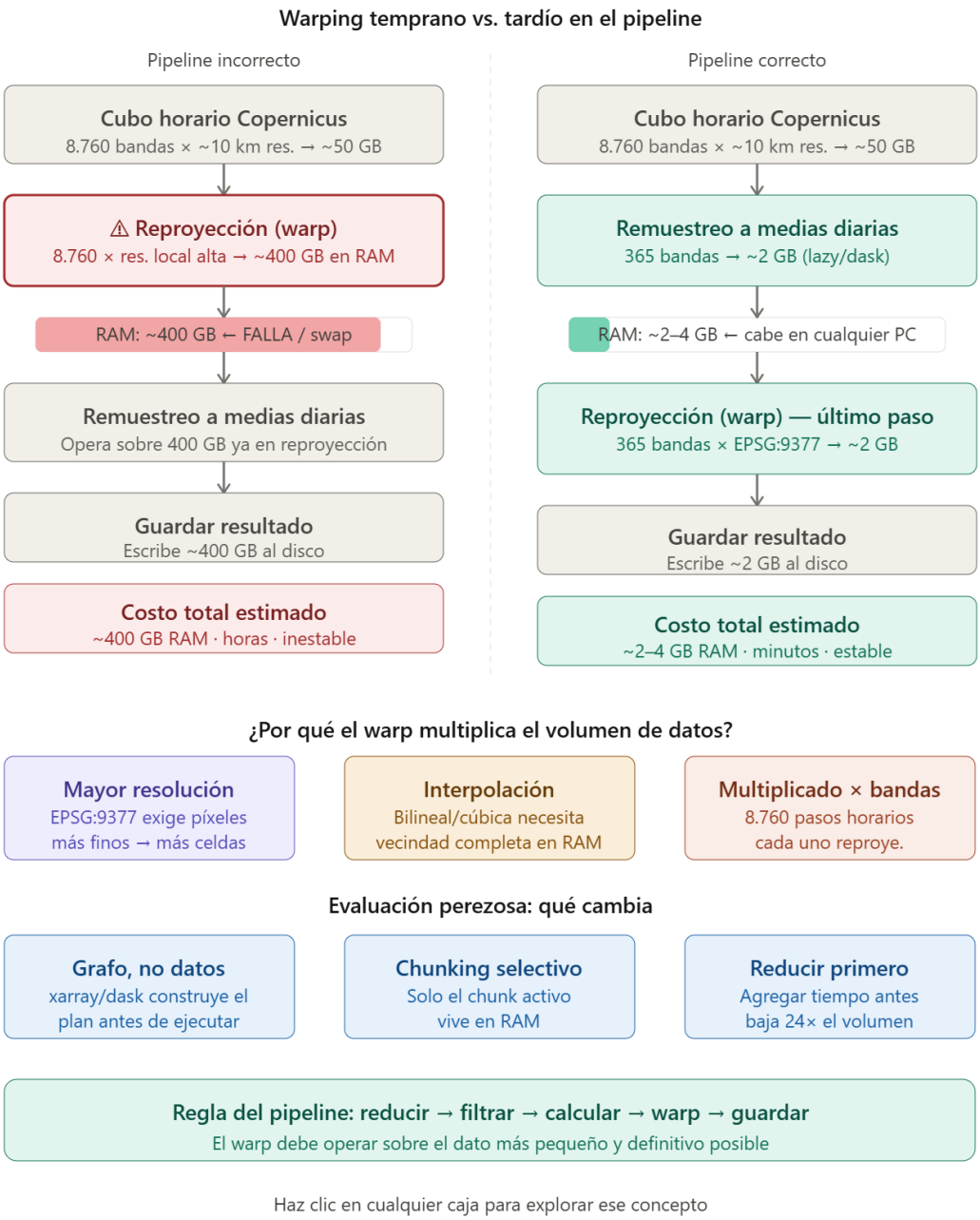


Figura 1: Representación del warping como proceso correcto e incorrecto

Justifiquen por qué el warping debe ser uno de los últimos pasos del pipeline.

El warping (reproyección) debe realizarse en los ultimos pasos del pipeline para redicir dimensionalidad:

La regla general es: reducir → filtrar → calcular → warp → guardar

Al remuestrear temporalmente primero (de horario a diario), se reduce el número de "rebanadas" del cubo. El warping es una operación costosa por píxel; aplicarla sobre menos píxeles es lógicamente más eficiente. Una operación de reducción temporal (media diaria) reduce el dato de ~400 GB a ~2–4 GB antes de que el warp lo vea. La diferencia no es de grado sino de categoría: 4 GB caben en la RAM de cualquier portátil; 400 GB ni siquiera caben en un servidor mediano sin swap agresivo.

Asimismo, el tiempo de un warp escala aproximadamente como O(n_bandas × n_píxeles_salida). Reducir primero de 8.760 a 365 bandas implica que el warp tarda ~24 veces menos, independientemente del método de interpolación. Un warp masivo que excede la RAM disponible fuerza al sistema operativo a usar swap en disco, lo que puede tardar horas y termina en OOM killer o corrupción silenciosa de datos.

Por lo anterior, el warp debe operar siempre sobre el dato más pequeño y definitivo posible.

2. Sobre Convenciones CF y Tiempo: Basados en su experiencia resolviendo el Ejercicio 2, ¿su lenguaje de programación decodificó automáticamente las fechas del archivo NetCDF al cargarlo, o tuvieron que realizar alguna operación manual para que el eje de tiempo fuera reconocido como fechas reales y no como simples números? Expliquen brevemente.

La decodificación de las fechas se realizo de manera automática con xarray, sin ninguna operación manual.

xr.open_dataset() tiene el parámetro decode_times=True activado por defecto. Al ejecutar la línea: *pythonds = xr.open_dataset(nc_file)*, xarray internamente realiza tres cosas:

- Lee el atributo "units" del eje de tiempo en el archivo .nc, por ejemplo "hours since 1900-01-01 00:00:00.0"
- Lee el atributo calendar, típicamente "gregorian" en archivos ERA5/Copernicus.
- Llama a *cftime.num2date()* internamente y convierte los números crudos a objetos de fecha reales.

Lo que se imprimio en el print(ds) mostró algo como esto: Dimensions: (time: 8760, latitude: 181, longitude: 360) Coordinates: * time (time) datetime64[ns] 2023-01-01 … 2023-12-31T23:00:00 * latitude (latitude) float32 90.0 89.0 … -90.0 * longitude (longitude) float32 0.0 1.0 … 359.0

El tipo datetime64[ns] en la coordenada time es la evidencia directa de que la decodificación fue automática. Si no hubiera ocurrido, se habrían visto algo comolo siguiente: * time (time) int32 0 1 2 3 … 8759

El único caso donde se habria necesitado intervención manual sería cuando el archivo usa un calendario no estándar como **360_day** (algunos modelos climáticos). En este caso, xarray no

puede convertir a *datetime64[ns]* de *numpy* y retorna objetos *cftime* en su lugar. En ese caso, operaciones como *.sel(time="2023-06")* requieren ajuste. Pero para archivos ERA5/Copernicus con calendario gregoriano estándar, la decodificación es completamente transparente.